Comprehensive Report: MNIST Classification Using ANN, SVM, and CNN

1. Introduction

This report presents a detailed analysis of three machine learning approaches for handwritten digit recognition on the MNIST dataset:

1. **Artificial Neural Network (ANN)**
2. **Support Vector Machine (SVM)**
3. **Convolutional Neural Network (CNN)**

The experiments were conducted in Google Colab using TensorFlow and scikit-learn, with systematic evaluation of model architectures and hyperparameters.

---

2. Dataset Preparation

2.1 Data Characteristics

- **Total samples**: 70,000 (60,000 training, 10,000 test)
- **Image dimensions**: 28×28 pixels (grayscale)
- **Classes**: 10 digits (0-9)

2.2 Preprocessing

Step	Description	Implementation
Normalization	Pixel values scaled to [0,1]	x_train/255.0
Reshaping	ANN/SVM: Flattened to 784D CNN: (28,28,1)	reshape(-1,784) reshape(-1,28,28,1)
Label Encoding	One-hot vectors (10D)	to_categorical(y_train,10)
Shuffling	Random training sample order	Built into Keras

---

3. Model Implementations & Results

3.1 Artificial Neural Network (ANN)

Architecture

```
model = Sequential([
    Dense(neurons, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

- **Optimizer**: Adam (lr=0.001)
- **Training**: 5 epochs, batch_size=32

Performance Comparison

Neurons	Accuracy	Train Time	Params	Key Observation
64	97.26%	25.51s	55,050	Baseline config
128	97.55%	24.68s	109,386	Optimal balance
256	97.30%	33.88s	218,058	Diminishing returns

3.2 Support Vector Machine (SVM)

Training Setup

- **Subsampled training**: 10,000 samples
- **Kernels tested**: Linear, RBF, Polynomial

Results

Kernel	Accuracy	Train Time	Test Time	Best For
Linear	91.70%	4.87s	6.52s	Speed
RBF	95.94%	7.93s	21.64s	Accuracy
Poly	95.15%	8.13s	8.47s	-

3.3 Convolutional Neural Network (CNN)

Base Architecture

```
model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),

    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
return model
```

- **Optimizer**: SGD (momentum=0.9)
- **Batch size**: 64

Hyperparameter Tuning

Epoch Optimization (lr=0.01)

Epochs	Accuracy	Train Time	Epoch Time
5	98.84%	28.80s	5.76s
10	99.03%	52.35s	5.23s
15	99.01%	62.45s	4.16s

Epochs	Accuracy	Train Time	Epoch Time
20	98.95%	91.46s	4.57s

Learning Rate Comparison (10 epochs)

LR	Accuracy	Convergence
0.1	98.39%	Fast
0.01	99.13%	Optimal
0.001	98.18%	Slow
0.0001	93.46%	Very slow

---

4. Comparative Analysis

4.1 Accuracy Comparison

Model	Best Accuracy	Parameters	Data Used
ANN	97.55%	~109K	Full
SVM	95.94%	-	10K subset
CNN	99.13%	~225K	Full

4.2 Computational Efficiency

Model	Avg Train Time	Scalability	Hardware Demands
ANN	~25s	High	Low

Model	Avg Train Time	Scalability	Hardware Demands
SVM	~7s (10K)	Low	High (full data)
CNN	~52s	Medium	Moderate

---

5. Key Findings

1. **CNN Superiority**:
    - Achieved highest accuracy (99.13%) with proper tuning
    - Automatically learns spatial hierarchies of features
    - Requires careful hyperparameter selection
2. **ANN Efficiency**:
    - 97.55% accuracy with minimal tuning
    - Faster training than CNN
    - Simpler architecture for prototyping
3. **SVM Limitations**:
    - Memory constraints limit training size
    - RBF kernel performs best but is slow
    - Not suitable for full MNIST scale

---

6. Recommendations

1. **Production Deployment**:
    - Use CNN with:
        - 10 training epochs
        - SGD optimizer (lr=0.01, momentum=0.9)
        - Data augmentation for potential improvement

2. **Rapid Prototyping**:
    - ANN with 128-neuron hidden layer
    - Adam optimizer for automatic learning rate adaptation
3. **Future Work**:
    - Experiment with deeper CNN architectures
    - Implement dropout/batch normalization
    - Test on extended datasets (e.g., EMNIST)

---

7. CNN with ReLU Activation and SGD Optimizer (Initial Test)

Model Architecture:

- Conv2D layer (32 filters, kernel size 3x3, activation=ReLU)
- MaxPooling2D (2x2)
- Conv2D layer (64 filters, kernel size 3x3, activation=ReLU)
- Flatten
- Dense layer (128 neurons, activation=ReLU)
- Output Dense layer (10 neurons, activation=Softmax)
- Training Configuration:
- Optimizer: SGD (learning rate = 0.01, momentum = 0.9)
- Epochs tested: [5, 10, 15, 20]
- Batch size: 32
- Activation: ReLU (for all layers except output)
- No Dropout used

Results:

- Best number of epochs: 15 (accuracy plateaued after that)
- Final Accuracy: 87.2%
- Accuracy first 5 epochs: 54.1%, 62.3%, 69.4%, 74.5%, 78.9%
- Number of parameters: around 1.2M

- Average training time per epoch: 12.5 sec
- Average test time per epoch: 1.3 sec

Observation:
Training longer than 15 epochs did not significantly improve performance and led to slight overfitting.

---

8. Testing Different Learning Rates

Learning rates tested: [0.1, 0.01, 0.001]

Observations:

- 0.1: Converged fast, but unstable loss and lower validation accuracy.
- 0.01: Balanced convergence and accuracy (best result).
- 0.001: Too slow, took longer to train and reached lower accuracy within given epochs.
- Best Learning Rate: 0.01, because it provided stable training and good generalization.

---

9. Testing Different CNN and FC Configurations

Tested 4 alternative models, varying:

- Number of CNN layers (1 to 3)
- Number of filters (16, 32, 64)
- Number of FC layers (1 to 3)
- FC layer sizes (64, 128, 256)

Best Architecture Identified:

Conv2D (32 filters) → MaxPooling2D → Conv2D (64 filters) → Flatten → Dense(128) → Dense(64) → Output

Performance:

- Accuracy: 88.9%
- Params: ~950K (better memory use)
- Average training time per epoch: 10.3 sec

Observation:
Using 2 CNN layers with a reduced FC layer size gave a good balance between accuracy and model size. 3 CNNs increased accuracy slightly but doubled training time with diminishing returns.

---

10. Batch Size Variation

Tested Batch Sizes: 32, 64, 128

Results Summary:

Batch Size	Accuracy	Train Time/Epoch	Test Time/Epoch
32	88.9%	10.3 sec	1.3 sec
64	88.5%	8.6 sec	1.1 sec
128	87.4%	7.1 sec	1.0 sec

Best Batch Size: 64, as it provided similar accuracy with faster training and less fluctuation in loss.

---

11. Activation Function Tests

Replaced all ReLU activations with:

- **Sigmoid**
- **Tanh**
- **Elu**
- **Softmax**

Activation Function	Final Test Accuracy	Validation Accuracy (Last Epoch)	Training Time (s)	Observations
Sigmoid	**64.06%**	**65.99%**	**87.17**	**Very slow convergence. Model took several epochs to start improving. Saturation issues and vanishing gradients likely affected learning.**

Activation Function	Final Test Accuracy	Validation Accuracy (Last Epoch)	Training Time (s)	Observations
Tanh	**24.38%**	**25.52%**	**90.83**	**Model failed to learn meaningfully. Minimal improvement over epochs. Vanishing gradient more severe due to full-range (-1 to 1) saturation.**
ELU	**84.72%**	**86.83%**	**86.63**	**Best overall performance. Fast convergence, low loss, and steady improvement. ELU mitigates vanishing gradients and allows for stronger non-linearity.**
Softmax	**82.46%**	**84.71%**	**89.75**	**Performed well but not designed as a hidden activation function. Still outperformed sigmoid/tanh, likely due to smoother gradient and better output separation.**

Conclusion:
ELU remains the best activation due to faster training and better performance.

---

12. Optimizer and dropout layer Tests

Tested optimizers:

- Adam
- RMSProp

Tested dropout location:

- after_conv
- after_flatten
- both

Tested dropout rate:

- 0.25

- 0.5

Results:

Optimizer	Dropout Location	Dropout Rate	Test Accuracy
Adam	after_conv	0.25	0.9833
Adam	after_flatten	0.25	0.9833
Adam	both	0.25	0.9846
RMSprop	after_conv	0.25	0.9826
RMSprop	after_flatten	0.25	0.9842
RMSprop	both	0.25	**0.9857**
Adam	after_conv	0.5	0.9830
Adam	after_flatten	0.5	0.9825
Adam	both	0.5	0.9828
RMSprop	after_conv	0.5	0.9840
RMSprop	after_flatten	0.5	0.9832
RMSprop	both	0.5	0.9831

Conclusion:
RMSprop with both dropout layer with rate 0.25 is the best with accuracy 0.9857

---

13. Final Model Summary

- CNN layers: 1
- FC layers: 3
- Activation: ELU
- Optimizer: RMSprop (default parameters)
- Epochs: 10
- Batch Size: 32

- Dropout: 0.25 both layer
- Final Accuracy: 98.57%
- Parameters: ~980K
- Avg Train Time/Epoch: 9.1 sec
- Avg Test Time/Epoch: 1.1 sec